
CSC 412 – Operating Systems

Programming Assignment 06, Fall 2025

Tuesday, November 11th, 2025

Due date: Sunday, November 23rd, 11:59pm.

1 What this Assignment is About

1.1 In memoriam John Horton Conway [1917-2020]

We are getting close to the fifth anniversary of the the death of Dr. John Horton Conway, who passed away on April 11th, 2020, from complications from COVID-19. John Conway was a British mathematician who did significant contributions to fields of mathematics such as number theory, knot theory, and game theory, but his most famous creation is the “Game of Life” (probably in 1970 or shortly earlier), one of the earliest examples of Cellular Automaton, and still the most popular to this day.

1.2 Objectives

This assignment is not one in which you start from scratch, but rather one in which your objective is to add multithreading to an existing code based that was not necessarily written with threads in mind. You are not meant to change the entire code. In fact, most of it will stay untouched. Part of your job is to figure out which parts you need to modify. The objectives of Prog 05 are for you to

1. Get some practice at modifying an existing code base, and in particular develop a “feel” for how much of the existing code base you must understand in detail and how much only requires very superficial, global understanding;
2. Multi-thread an existing application;
3. Use external libraries in a C++ project;

(1 and 3: Yes, this is definitely very much part of my devious plan to use this course to *also* serve as a sort of “applied Software Engineering” course, the often-discussed and never implemented CSC306).

1.3 Handout

The main handout for this assignment is an implementation of a cellular automaton that uses OpenGL + glut for its front end. This program implements several rules, including the classical rule of Conway’s original Game of Life.

1.4 For a full-fledged implementation

My simple handout is only a quick & dirty implementation (in fact, it is based on a demo I did in a class a few years ago), using the same basic OpenGL + glut framework that we are going to use for assignments until the end of the semester. For a full implementation and some cool examples of cellular automata, download and run [Golly](#).

1.5 OpenGL and glut

1.5.1 What are OpenGL and glut?

This handout, as-is, requires the OpenGL and glut libraries. OpenGL is a library for doing 2D and 3D graphics. OpenGL is blissfully OS-unaware: It has no notion of time. It is equally ignorant of the things called windows, menus, mouse, and keyboard. All it does is render a scene into a buffer. If that buffer happens to be attached to a window, then you will see a rendering of the scene on your display, but OpenGL would be equally happy to render into a buffer attached to the file system or to an output device such as a printer.

Almost every GUI library (including the native GUI library of your OS of choice) provides functions to assign a window's buffer as the place for OpenGL to do its rendering into. Keyboard, mouse, and timer events can be handled either through the GUI's API or through regular system calls. The problem is that native GUI libraries are not portable and that multi-platform libraries (e.g. Qt or wxWidgets) are fairly complex.

This is where glut (which stands for GL utility toolkit) comes in. It was created as a minimalist, very simple library for providing some OS services to OpenGL:

- create and resize windows;
- handle keyboard events;
- handle mouse events;
- handle timer events;
- create drop menus.

A major advantage of glut is that almost exactly the same code can be built and run on different platforms, with only slight cosmetic difference. What differs between different platforms is the location of the headers and binaries for OpenGL and glut. This can be solved by using a header that defines these paths based on the OS and development environment for which the program is built. Some open-source libraries build on top of glut to offer more advanced GUI widgets such as buttons, sliders, file selectors, etc. An example of such a library is `pui`, which is distributed as part of the `plib` library.

One minor disadvantage of glut is that the interfaces built with glut (including the ones built on top of glut) are ugly¹. However, glut was never meant to build commercial-grade applications, just small demos and programming assignments for courses, so this drawback is fairly minor.

¹This may not be obvious to Linux users, because they don't particularly stand out as being significantly worse than other Linux apps, but on Mac or Windows, the difference is striking.

The main drawback of glut is that, on Mac and Windows at least, it is still a 32-bit library, now deprecated. It should be replaced by freeglut, which is theoretically 100% compatible with the original glut, the only difference being the name of the header file and binary². On Linux, freeglut has completely replaced glut, so that nothing has to be changed in the code nor in the paths.

1.5.2 About glut callback functions

Part of your job is figuring out how much/little you actually need to understand about the operation of the entire program. Don't try to work your way through all parts of the program trying to make sense of what is going on. For example, the actual drawing of Pacman and ghosts are parts that you don't need to bother with. Same with the interface, except for one crucial aspect: the glut callback functions and glutMainLoop. I will try to explain here briefly the key elements that you need to understand.

The programmer sets up a glut interface by defining “callback functions.” You essentially tell glut

- “This is the function to call whenever you need to render the scene” (set by a call to glutDisplayFunc);
- “This is the function that you should call whenever the user hits a key on the keyboard” (set by a call to glutKeyboardFunc);
- “This is the function to call whenever the user does something with a mouse button” (set by a call to glutMouseFunc);
- “This is the function to call if the user tries to change the window's width or height” (set by a call to glutResizeFunc);
- “This is the function to call after a fixed amount of time” (set by a call to glutTimerFunc);
- And a few more for people interested: glut has a primitive pop-up window system, can track mouse movements, etc.

After defining all these callback functions, the program relinquishes all control to glut by calling the function glutMainLoop. It is glut that will call the drawing function, will test the keyboard, etc., and will call the callback functions. This works generally well, but threads add a layer of complexity:

- It is essential that the thread that calls glutMainLoop be the main thread. Nothing will work otherwise.
- You must create your threads before the call to glutMainLoop.
- Nothing will get drawn (in fact, on some platforms, the window may even not appear) before the call to glutMainLoop.
- When the main thread is blocked on a lock, the display will not refresh, and the application will not react to keyboard and mouse events.

²This being said, in recent versions of mac OS, I have had serious problems with freeglut, to the point that I have decided to revert to Apple's deprecated version of GLUT).

1.5.3 Install MESA and FreeGlut on Ubuntu

If you are on macOS, then you don't need to install anything: OpenGL and glut are part of the system (although they are officially tagged as “deprecated” since Apple is trying to push developers to use its new-ish, much lower-level 3D framework named Metal).

On Linux (or WSL), we are going to use MESA, which is an open-source implementation of OpenGL, the only free option on Linux. MESA and FreeGLUT come installed by default on Ubuntu, so that you can run pre-built OpenGL, but in order to build a MESA project, you need to install the *developer* version of both libraries.

Make sure first that your Ubuntu install is up to date, and then execute:

- `sudo apt-get install freeglut3-dev`
- `sudo apt-get install mesa-common-dev`

1.5.4 How to build the handout

If you are on Linux, then you should be able to build the handout as follows:

```
g++ -Wall -std=c++23 main.cpp gl_frontend.cpp -lGL -lglut -o cell
```

On macOS, OpenGL and glut come as *frameworks* (which is the way Apple organizes individual libraries into a single folder containing the dll, static library, and the header files). So the build code is slightly different:

```
g++ -Wall -std=c++23 main.cpp gl_frontend.cpp -framework OpenGL  
-framework GLUT -o cell
```

2 Part II: Cellular Automaton

2.1 Cellular automata

2.1.1 Concept

Cellular automata (CA for short) were developed as a model for the way cells or organisms develop and die. The basic idea is that if you are all alone in a desert you die, and if you are too crowded you die as well. In other case you can keep going or, even better, prosper and create new life.

The most famous cellular automaton is the Game of Life of British mathematician John Horton Conway:

<http://zh-yue.wikipedia.org/wiki/%E7%94%9F%E5%91%BD%E6%A3%8B>
http://www.conwaylife.com/wiki/Cellular_automaton

The reason why cellular automata are so interesting for mathematicians (and programmers) is that with extremely simple rules it is possible to produce very complex dynamic behaviors such as patterns that repeat over time (called “oscillators”), “glider” patterns that move over the screen (Figure 1), etc. Some of these patterns can be quite large, like the ones nicknamed “spaceships.”

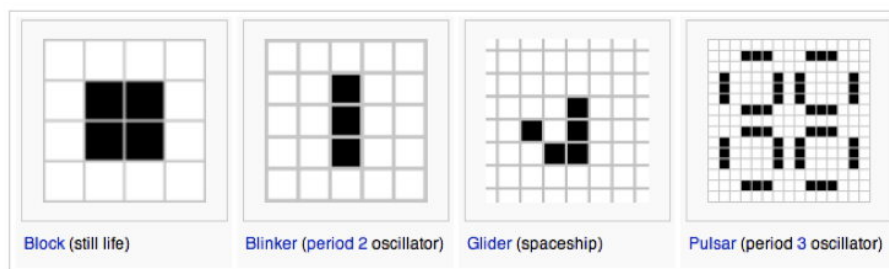


Figure 1: Examples of repeating patterns in the Game of Life (from Wikipedia)

2.1.2 How it works

In their simplest form, cellular automata are represented as rectangular grids of “cells” that are either dead or alive. At each new generation of the cellular automaton, each cell looks at all of its immediate neighbors and counts how many of them are alive. A live cell that has too few live neighbors will die (desert). So will a live cell with too many live neighbors (overcrowding). Otherwise a live cell will stay alive. Life can restart at a “dead” location if the right number of live neighbors are present.

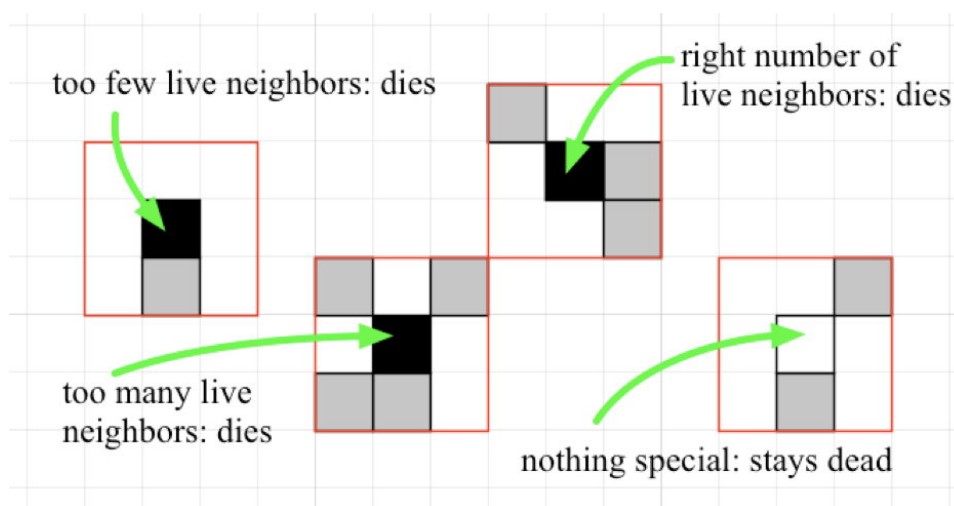


Figure 2: How cells live or die in the Game of Life.

Different cellular automata differ in the rules that specify under what conditions a cell will die or come to life. The rules for CAs are generally presented in the form Bxxx/Syyy. For example, the classical Game of Life is B3/S23. What does this mean?

- A dead cell with exactly 3 live neighbors will get **B**orn again (alive).
- A cell that is alive will **S**tay alive if it has either 2 or 3 neighbors alive (if it has 0, 1, 4, 5, 6, 7, or 8 neighbors alive, then the cell will die).

Figure 2 shows some examples of cell birth/death. In the figure I have drawn “alive” cells in black or gray, and “dead” cells in white. We will do the reverse in our application because it will look prettier when we start using color to represent the “age” of live cells.

Note: The basic CA have only two states for their cells, but it is of course possible to define more states, and rules that take this into account. For example, the CA known as “Brian’s Brain” has cells that can be “alive,” “dead,” or “dying” (a cell does not go directly from “alive” to “dead” without one generation in the state “dying,” which is a form of “second chance” algorithm).

2.2 Now look at the code

Generally, you should understand well all that is going on in `main.cpp`, but you only need to have a general idea of what is going on in `gl_frontend.cpp`. You shouldn’t have to change much in the callback functions (and in particular in `gl_frontend.cpp`), except

- in the keyboard event-handling callback function, to take care of the slowdown-speedup functionality discussed later;
- in the timer callback function, there is currently a call to `threadFunc` that must *absolutely* go away when you start multithreading the application. If that call stays there, you will get all sorts of odd behaviors.

2.3 What you should be aware of (and careful about)

2.3.1 The glut thread must be the main thread

I remind you that glut was created as a minimalist, very simple library for providing some OS services to OpenGL:

- create and resize windows;
- handle keyboard events;
- handle mouse events;
- handle timer events;
- create drop menus.

In order to keep glut simple, its designer imposed some restrictions. The most important for us is that the glut thread (the thread that makes the call to `glutMainLoop()`, and thereafter surrenders control of calls to glut, must be the main thread, that is, the thread that got created when the process was launched. It *cannot* be a thread created by `pthread_create` (or a C++ thread or task object created in your program).

The basic philosophy of a multithreaded OpenGL-glut application is that the main thread, besides handling keyboard and mouse events, “takes pictures” at a given frequency. That is, every so-many milliseconds, it calls the display callback function to draw whatever is the current state of the different objects in the simulation.

The computation threads (in this application, the threads that each a range of rows of the cellular automaton) only take care of their own business: They update their part of the grid and do nothing else. In particular, they do not make calls to force the display. This is simply not their job.

2.3.2 Keep the threads in phase

Right now, there is a single thread in the process: the main thread which then dedicates itself as rendering thread once it makes the call to `glutMainLoop()`. That thread is the one running the call to compute the next generation.

Now that you are going to have multiple computation threads, how are you going to know that the “next grid” has been completely computed and that we can now safely swap the grids and launch the computation of the next generation? We are going to do this in two different ways (and a third one for extra credit).

3 What to Implement

3.1 Organization

Since the different versions *may* impose that you make different changes to the different source and header files, we are simply going to organize each version into its own folder named `Version1`, `Version2`, `EC1`, etc. Each version folder will contain two subfolders:

- A folder named `Program` containing all the source and header files required to build this version of the program;
- A folder named `Scripts` containing a build script `build.sh` for this version of the program and whatever script is explicitly required for the version.

New in the build script:

We haven’t done any OS-specific bash scripting yet, and today is our chance to do it. Your build script should determine whether the script is run on macOS or Linux (including WSL on Windows) and choose the build command to run accordingly.

3.2 Version 1: Short-lived threads

3.2.1 The idea

In this version, threads will only last for one generation of the Cellular Automaton. The main thread creates the required number of threads, and joins them at the end of their computation.

Note that the different parameters of each thread remain pretty much the same from one generation to the next. This means that the `ThreadInfo` array should only be allocated once, and should be used over and over by the different generations of threads.

3.2.2 First, modify the main function to take arguments

In the handout, the number of rows and columns of the grid are defined as constants. You should modify the code so that the program now takes three arguments: the width (number of columns) and height (number of rows) of the grid, as well as the desired number of computation threads of the program. You should verify that your program indeed received three arguments, that these are strictly positive integers, with the width and height larger than 5, and that the number of threads is not larger than the height of the grid.

Style-wise, this means that the constants `NUM_ROWS`, `NUM_COLS`, and `MAX_NUM_THREADS` won't be constants anymore and should be renamed, from the current all-caps-separated-by-underscores forms to lowercase with internal capitalization, since they are going to become regular variables.

3.2.3 Multithread the program

The simplest/best way to do this is to assign a horizontal band of the grid to each process. Of course, depending on the number of threads and the height of your grid, the last computing thread may not have the same number of rows to process as the other threads.

Because we read the data into the “current” version of the grid and write the results into the “next” copy of the grid, and no two threads ever write at the same location in the “next” grid, there is no synchronization problem between computation threads.

On the other hand, there is a problem with the rendering thread (which is the main thread of the process, once it engages in the call to `glutMainLoop()`). In theory, there could be a race condition on the `nextGrid` and `nextGrid2D` variables.

3.2.4 How to split the work?

We did something like that in the last lab: Each thread should work on a separate range of grid rows. Just make sure that you split the work as evenly as possible. So, for example, if you want to split 13 rows between 4 threads, 4-3-3-3 is a good split, but 4-4-4-1 is not.

As we discussed in a lab, the way to do that is to assign to each thread a start row and an end row indices. Of course, what better place to keep this information than the thread struct that you now—hopefully—automatically define whenever you want to implement a multithreaded solution.

3.2.5 One last thing: speedup and slowdown

Finally, for this version to be complete, you need to figure out a way to enable the speedup and slowdown key controls. This does not mean adjusting the delay of the callback to the timer function (which should be set to a value around 10 to 30 milliseconds). Rather, you want to adjust the sleep time between generations.

3.2.6 Extra credit 1 (3 pts)

After ranting on and on in class about the horrid C pseudorandom number generator, you may be surprised to see me use it in the handout code. I'll concede that the C generator is handy enough for junk code that is not meant to pass the night, and I used it in that spirit. Well, here is your chance to set me straight and use a proper C++ generator instead. For this generation of a random live/dead state

for the grid cells, I recommend that you take a look at the `std::bernoulli_distribution` class.

3.3 Version 2: Persistent threads

3.3.1 What's the same as in Version 1

Just as in Version 1, the program should take the number of rows, columns, and computation threads as argument. The speedup/slowdown functionality should be implemented as well.

3.3.2 What's new

In this version, the threads are going to keep running until the program quits. The problem we have to take care of now is that we need to wait until all the threads have completed their work on their respective range of rows before we swap the “current” and “new” grids.

3.3.3 How we do it

We are simply going to add one or more `bool` fields to our `threadInfo` struct so that a thread can indicate whether it has finished its work, and also be told when it can start working on a new generation.

Although there is no risk of a race condition if this is implemented properly (the “main thread” will have to play a role in this), this is not a pretty solution. We will work on a lock-based solution in the next version.

3.4 Version 3: Persistent threads with locks

This solution will require multiple locks. The idea is that, when a computation thread has completed the work on its range of rows, it is going to check whether it is the last thread to complete its work (for this generation):

- If it is not, then it should put itself to sleep, and wait to be awoken;
- If it is the last to finish, then it know that all the other computation threads are sleeping. So it can first swap the grids, and then wake up the other threads so that all can start working on the next generation.

3.5 Version 4: Randomized, in-place implementation

3.5.1 The concept

Instead of a dumb execution in artificial, strict order, row-by-row and column-by-column, why not let the update be more natural and random? In this version, each of the computation threads would randomly select a cell to update and would perform the update *in place*. In other words, there wouldn't be anymore a “next grid,” as all calculations would be done directly on the “current grid.” This also means that all cells won't be anymore all at the same generation. Some cells will “age” faster than others if they are lucky, or unlucky, depending how you see it.

Note that for this version to work properly, and you to see the display change at all, you will need to create a lot more threads than in the previous versions. You may also need to set a shorter sleep time for the threads.

3.5.2 Synchronization issues

Obviously this also means that the solution you worked out for the “last thread to process its assigned range of grid rows” problem will not work here: A processing thread is never done. There is always a new location to update.

A proper solution would involve a 2D array of mutex locks, one for each cell of the grid (so, essentially, we replace one grid, the “next grid,” by another: the array of mutex locks). In order to update the state of one cell, the computing thread would need to acquire the nine locks of the 3×3 square centered at that cell.

For this section of the assignment, we are going to ignore the synchronization problem altogether, and accept that our implementation is going to run into race conditions all the time.

3.5.3 Extra credit 2: Discussion on synchronization (up to 6 points)

Now, synchronizing access to the entire grid with the rendering thread is even more critical than before. Here again, I let you think about it and propose your own solution in your report. I will just give you a hint: Does this look somewhat like one of the classical synchronization problems?

4 Extra Credit 3: bash Script (12 points)

4.1 Control your program from a bash script

4.1.1 What it's about

The program of the handout uses a very rudimentary GUI to specify some of the parameters of the cellular automaton. Using keystrokes, the user can select the cell reproduction rule, speedup or slowdown the simulation, switch the display from black and white to color, and also terminate execution.

What I want you to do is to allow the user to perform the same adjustments to the cellular automaton by giving commands to the script. The purpose of this task in the assignment is not to improve the user experience for this particular program but simply to add one more tool to your toolbox, even if the task we use it for here is borderline silly/useless.

4.1.2 What the script should do

The script should (not necessarily in this order):

- Get as arguments the width and height, and number of threads of the cellular automaton process to launch;
- Launch a cellular automaton process, specifying the desired width and height;

- Setup a named pipe to communicate with the cellular automaton process, using the command syntax defined in the next subsection.

4.1.3 Syntax of the commands

After the script has launched the cellular automaton process, it should take commands from the user (and send these commands to the process using a named pipe). The syntax of these commands is:

- `rule <rule number>` selects the automaton's cell reproduction rule. For example, `rule 2` selects the "coral growth" rule.
- `color on` enables the color mode;
- `color off` disables the color mode (reverts to black and white);
- `line` toggles on-off the display of lines for the grid of cells;
- `faster` increases the speed of the simulation;
- `slower` reduces the speed of the simulation;
- `end` terminates execution of the cellular automaton of the process and of the script.

Invalid commands (including rule selection commands with an invalid rule number) should be properly rejected without terminating execution.

4.2 Extra credit 4 (up to 8 points)

Modify your script so that it becomes a pure command interpreter and even the launching of a cellular automaton process is done via a script command. This means that your script could now launch and control multiple cellular automata. The command to launch a new cellular automaton would be

- `cell width height`

Each new cellular automation process would receive an index (starting with the first process at an index of 1) and display that index in its window's title.

All other commands to be interpreted by the script should now be prefixed by the index of the cellular automaton process to which they should be addressed:

- `3: faster` means that Cellular automaton 3 should increase its speed;
- `2: end` means that Cellular automaton 2 should terminate execution;
- etc.

Invalid commands, including commands meant for a process that hasn't been created or one that has already terminated, should be rejected.

5 Some other extra credit options

5.1 Extra credit 5: Implement a couple more rules (2 points)

The Golly application comes with a large number of rules that result in cool behaviors. Implement a couple of them.

5.2 Extra credit 6: Load some predefined automata (5 points)

Golly also includes some initial states configurations for the cellular automata. Add to the program the ability of reading one of these `.rle` files.

6 What to Hand In

6.1 Organization of the submission

Create a new folder named `Prog06`. In that folder, place

- subfolders after all the versions and extra credit that you implemented: `Version1` and `Version2`, etc.;
- your report as a `.pdf` file.

6.2 To verify before you submit the assignment

I should be able to run your code “as is” on my own virtual machine. In particular, I should not have to make changes/fixes to your code for it to build or run.

Important note on the C++ program: Your C++ program *must* build with no errors. If there are compiler errors, then you will get a grade of 0 for the code and execution sections of the grading.

Relative paths: Make sure that your C++ code and `bash` script don’t contain any hard-coded paths. To verify this, create a new folder on your home directory, make a copy of the C++ source file and `bash` script there, rebuild your program, and verify that the script runs fine in the new location. It is *your* job to make this kind of verifications before you submit an assignment.